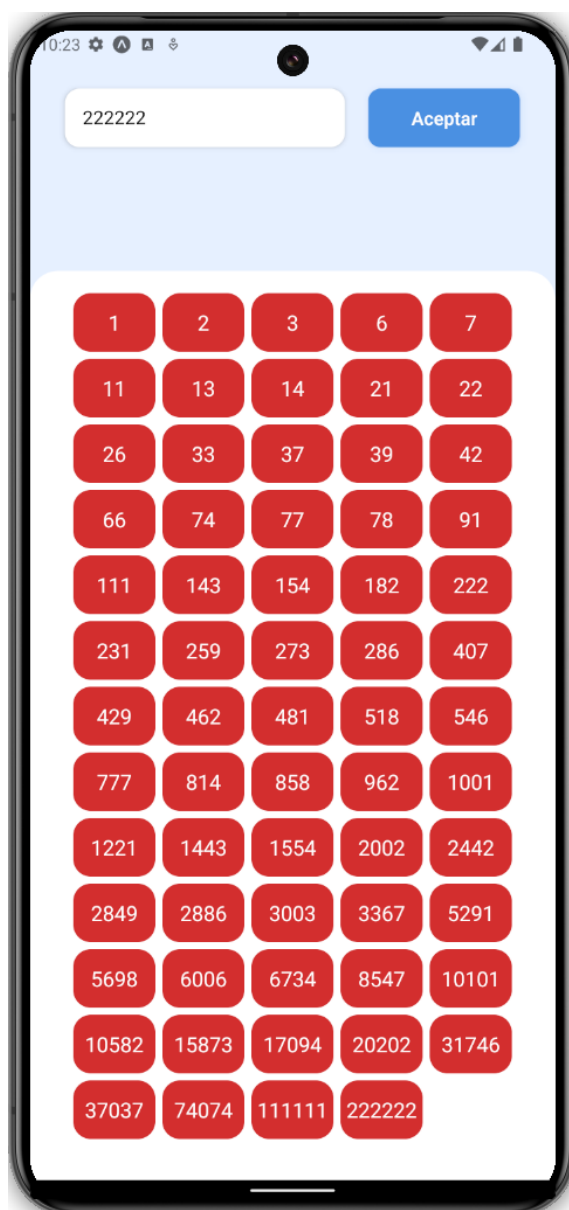


TUTORIAL 12

MODALES



En este tutorial vamos a aprender a usar los **modales**, que son pestañas que se superponen a la aplicación, de manera que es imposible volver a ella sin haber cerrado el modal. Dentro del modal, se muestra un componente, como por ejemplo, un formulario.

1.- Inicio del proyecto

Este tutorial comienza en el punto donde terminó el anterior, así que tendremos que abrir Visual Studio Code con el proyecto que ya teníamos

1. Abre el proyecto **rn_divisores**
2. Crea una rama llamada **tutorial12**
3. Borra el **View** del contenedor secundario (antes copia en el portapapeles el **FlatList**), de forma que la interfaz quede solo con la fila para introducir datos

```
1. export default function App() {
2.   // inicio omitido
3.   return (
4.     <View style={styles.contenedorPrincipal}>
5.       <View style={styles.fila}>
6.         <TextInput
7.           style={styles.cuadroTexto}
8.           placeholder="Escribe un número positivo"
9.           placeholderTextColor="#666"
10.          value={texto}
11.          onChangeText={setTexto}
12.        />
13.        <Pressable
14.          style={{pressed}} => pressed? [styles.boton,{opacity:0.4}]:styles.boton}
15.          onPress={aceptarPulsado}>
16.          <Text style={styles.textoBoton}>Aceptar</Text>
17.        </Pressable>
18.      </View>
19.    </View>
20.  )
21. }
```

2.- Modales

Vamos a hacer que al pulsar el botón de aceptar, se abra un **modal** que muestre la tabla con los divisores del número introducido.

Un **modal** es una pestaña que se sitúa por encima del componente principal y aparece con una animación (normalmente, aparecer desde abajo). El modal muestra en su interior otro componente.

Es importante indicar que el modal se ve siempre, así que habrá que introducir una variable de estado llamada **modalVisible** que indique si hay que dibujar o no el modal mediante **renderizado condicional** (hay otra forma más sencilla de mostrar/ocultar el modal mediante un prop, pero puede dar lugar a problemas)

El **modal** es un componente llamado **Modal** cuyos principales props son:

- **animationType:** Indica cómo aparecerá el modal en la pantalla. Sus valores son:
 - o **slide** → El modal entra en la pantalla desde abajo
 - o **fade** → El modal aparece y se pone encima de la pantalla
 - o **none** → No hay animación y el modal se sitúa encima de la pantalla
- **transparent:** Si es **true**, el modal tiene fondo transparente y se seguirá viendo la pantalla principal. Es lo que se hará casi siempre, para así controlar la apariencia del modal (si se deja a **false**, el modal tiene un fondo por defecto)

1. En **App.tsx**, añade un prop llamado **modalVisible**, inicializado a **false**, y su setter se llamará **setModalVisible**

```
1. export default function App() {  
2.   const [modalVisible, setModalVisible] = useState(false)  
3.   // resto omitido  
4. }
```

2. Haz que dicho prop valga **true** en el momento de asignar la lista de divisores

```
1. function aceptarPulsado(){  
2.   const {exito,valor} = toEnteroPositivo(texto)  
3.   if(exito){  
4.     const lista = calcularDivisores(valor)  
5.     setListaDivisores(lista)  
6.     setModalVisible(true)  
7.   }else{  
8.     Alert.alert("Error","Debe introducirse un número entero positivo")  
9.   }  
10. }
```

3. Donde antes estaba el contenedor secundario, abre un bloque de renderizado condicional, de manera que si **modalVisible** vale **true**, se muestre un componente **Modal** con los siguientes props:

- **animationType**: slide
- **transparent**: false

```
1. export default function App() {  
2.   // resto omitido  
3.   return (  
4.     <View style={styles.contenedorPrincipal}>  
5.       <View style={styles.fila}>  
6.         <TextInput  
7.           style={styles.cuadroTexto} placeholder={"Escribe un número positivo"}  
8.           placeholderTextColor={"#666"}  
9.           value={texto}  
10.          onChangeText={setTexto}  
11.        />  
12.        <Pressable  
13.          style={{pressed}} => pressed? [styles.boton,{opacity:0.4}]:styles.boton  
14.          onPress={aceptarPulsado}>  
15.          <Text style={styles.textoBoton}>Aceptar</Text>  
16.        </Pressable>  
17.        {  
18.          modalVisible && (  
19.            <Modal animationType={"slide"} transparent={false}>  
20.              </Modal>  
21.            )  
22.          }  
23.        </View>  
24.      </View>  
25.    )  
26.  }
```

4. Dentro del componente **Modal** pon el **FlatList** que muestra la lista de divisores

```
1. <Modal animationType={"slide"} transparent={false}>  
2.   <FlatList  
3.     data = {listaDivisores}  
4.     renderItem={CajaDivisor}  
5.     keyExtractor={ numero => numero.toString()}  
6.     numColumns={5}  
7.   />  
8. </Modal>
```

5. Ejecuta la app y comprueba que al introducir un número aparece desde abajo una pestaña que contiene la tabla de primos

Como **modalVisible** vale **true** y no hay ningún momento del código en que cambie de valor, no se puede cerrar el modal. La única forma es reiniciar la app

6. Para poder cerrar la app, provisionalmente vamos a hacer que al pulsar en la lista de divisores, este se cierre. Basta con encerrar el **FlatList** en un **Pressable** que al ser pulsado, ponga a **false** la variable **modalVisible**

3.- Dar estilo al modal

Como podemos ver, el modal ocupa toda la pantalla y por defecto está formado por una pantalla en blanco sobre la que se sitúa el componente que se muestra en el modal. En este punto vamos a cambiar su apariencia.

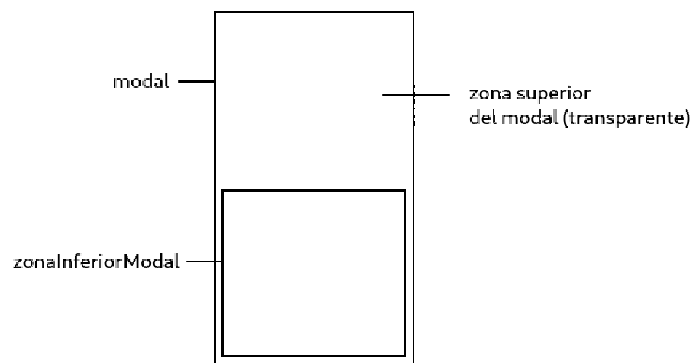
Para controlar la apariencia del modal, primero hay que cambiar su prop **transparent** y ponerlo a **true**. De esa forma, el modal estará formado por una pantalla transparente (se ve el componente sobre el que se sitúa el modal) y podremos poner en el interior de la etiqueta **Modal** nuestro diseño.

Vamos a hacer que el **Modal** consista en una pestaña de esquinas redondeadas que aparezca por abajo, y que se ajuste al **FlatList**

1. Crea el siguiente estilo, para dar forma a la pestaña del modal:

```
1. zonaInferiorModal: {
2.   position:"absolute", // posicionamiento absoluto para anclar esta parte al fondo del modal
3.   bottom:0, // situará la pestaña en el fondo
4.   width:"100%", // ocupará todo el ancho del modal
5.   backgroundColor: "#fff", // color de fondo blanco
6.   borderTopLeftRadius:24, // esquinas superiores redondeadas
7.   borderTopRightRadius:24,
8.   paddingHorizontal:24, // relleno por lo laterales
9.   paddingTop:16,
10.  paddingBottom:32,
11.  alignItems:"center", // centrado del FlatList
12. },
```

☞ **¿Por qué el posicionamiento absoluto?** Aunque no lo parezca, el modal siempre ocupa toda la pantalla, y lo que estamos haciendo es dividirlo en dos partes: una zona inferior que tiene el estilo indicado, y una zona "superior", que es la parte que queda por fuera. El posicionamiento absoluto sirve para colocar el **View** en el fondo de la pantalla del modal.



2. Vamos a hacer que al pulsar en la zona exterior se cierre el modal. Para ello, borra el **Pressable** que rodeaba al **FlatList**

```
1. <Modal animationType="slide" transparent={true}>
2.   <View style={styles.zonaInferiorModal}>
3.     <FlatList
4.       data={listaDivisores}
5.       renderItem={CajaDivisor}
6.       keyExtractor={(numero) => numero.toString()}
7.       numColumns={5}
8.     />
9.   </View>
10. </Modal>
```

3. A continuación, añade dentro del **Modal**, pero antes del **View** de la zona inferior, un **Pressable** que se expanda ocupando el resto de espacio libre del modal y que al pulsarlo ponga a **false** la variable **modalVisible**

```
1. <Modal animationType="slide" transparent={true}>
2.   <Pressable onPress={() => setModalVisible(false)} style={{flex:1}}/>
3.   <View style={styles.zonaInferiorModal}>
4.     <FlatList
5.       data={listaDivisores}
6.       renderItem={CajaDivisor}
7.       keyExtractor={(numero) => numero.toString()}
8.       numColumns={5}
9.     />
10.   </View>
11. </Modal>
```

4. Ejecuta la app y comprueba que puedes cerrar el modal pulsando “fuera” de él.

En realidad no estás pulsando fuera del modal, sino que estás pulsando en su parte superior, que es transparente

5. Vamos a destacar más la parte exterior, poniéndole de fondo un gris con opacidad. Comienza creando el estilo para la parte superior

```
1. zonaSuperiorModal:{
2.   flex:1,
3.   backgroundColor:'rgba(0,0,0,0.1)'
4. },
```

6. Pon ese estilo al **Pressable**

```
1. <Modal animationType="slide" transparent={true}>
2.   <Pressable onPress={() => setModalVisible(false)} style={styles.zonaSuperiorModal}/>
3.   <View style={styles.zonaInferiorModal}>
4.     <FlatList
5.       data={listaDivisores}
6.       renderItem={CajaDivisor}
7.       keyExtractor={(numero) => numero.toString()}
8.       numColumns={5}
9.     />
10.   </View>
11. </Modal>
```

Oscurecer la zona transparente es buena idea para dar la sensación de que el componente ocultado es inaccesible y que el usuario no puede interactuar con él.

7. Haz un commit titulado “Finalizado tutorial 12”

8. Sitúate en la rama **main**
9. Mezcla la rama **tutorial12** en la rama **main**
10. Haz un **push**

4.- Limitaciones del componente Modal

*Como hemos visto en este tutorial, el componente **Modal** incluido en React Native es muy útil y sencillo, pero desgraciadamente tiene limitaciones importantes y para apps más complejas no es adecuado:*

- *Ocupa siempre toda la pantalla y por tanto, no sirve para apps donde queramos una pestaña inferior auxiliar y podamos seguir interactuando con la pantalla*
- *Admite muy pocas opciones de personalización y solo tiene tres posibilidades: slide, fade y none*
- *El efecto de opacidad para la zona exterior no queda bien, porque la parte transparente “sube”. Se solucionaría haciendo que el modal sea entero transparente, y con una librería de animaciones, mover la pestaña desde abajo hasta su posición.*
- *Puede dar problemas con el **SafeAreaView***
- *Tampoco se comporta bien con el teclado: es frecuente que al aparecer un teclado, el contenido del modal “suba” y tape cuadros de texto*
- *No se pueden poner múltiples modales a la vez*

*Cuando la presencia de estos inconvenientes es un problema, es necesario utilizar librerías como **react-native-modal**, que ofrecen modales más avanzados.*